

ELEVANCE HEALTH

Commingle Project

Member Deductible & Out-of-Pocket

Accumulator Synchronization Platform

End-to-End Architecture & Implementation Guide

Attribute	Details
Project Name	Elevance Commingle — Accums Sync
Architecture	AWS S3 + Snowflake Medallion (Bronze / Silver / Gold)
Data Integration	IAM Role-Based S3 to Snowflake External Stage
Data Format	CSV (Inbound & Outbound)
Vendors	Pharmacy, Dental, Mental Health Partners
Document Version	v1.0
Classification	Internal — Confidential

1. Executive Summary

Elevance Health is a leading US-based healthcare insurance company that provides comprehensive medical coverage to its members. For non-medical services — including pharmacy, dental, and mental health — members are served by a network of external vendor partners contracted with Elevance.

The Commingle Project addresses a critical operational challenge: ensuring that a member's Deductible and Out-of-Pocket (OOP) accumulator balances remain synchronized across Elevance's internal systems and all participating external vendors in real time. Discrepancies in accumulators can result in members being overcharged, vendors applying incorrect cost-sharing, or regulatory non-compliance.

Business Problem

When a member uses pharmacy services with a third-party vendor, that vendor accumulates charges toward the member's deductible. Without synchronization, Elevance's internal records may differ from what the vendor has on file, causing billing disputes, incorrect claims adjudication, and poor member experience.

The Commingle platform establishes a bi-directional data pipeline to keep all accumulators in sync, with full audit trails and reconciliation capabilities.

1.1 Project Objectives

- Synchronize member Deductible and OOP accumulators between Elevance and all non-medical vendors
- Ingest CSV data from vendors (Inbound) into a governed Snowflake data platform
- Export medical accumulator data from Snowflake to vendors (Outbound)
- Implement Medallion Architecture (Bronze → Silver → Gold) for data quality and governance
- Use AWS IAM roles for secure, credential-free S3 access from Snowflake
- Provide complete audit trail, reconciliation, and error handling for every file processed
- Build data quality checks and SLA monitoring dashboards

1.2 Scope of Services

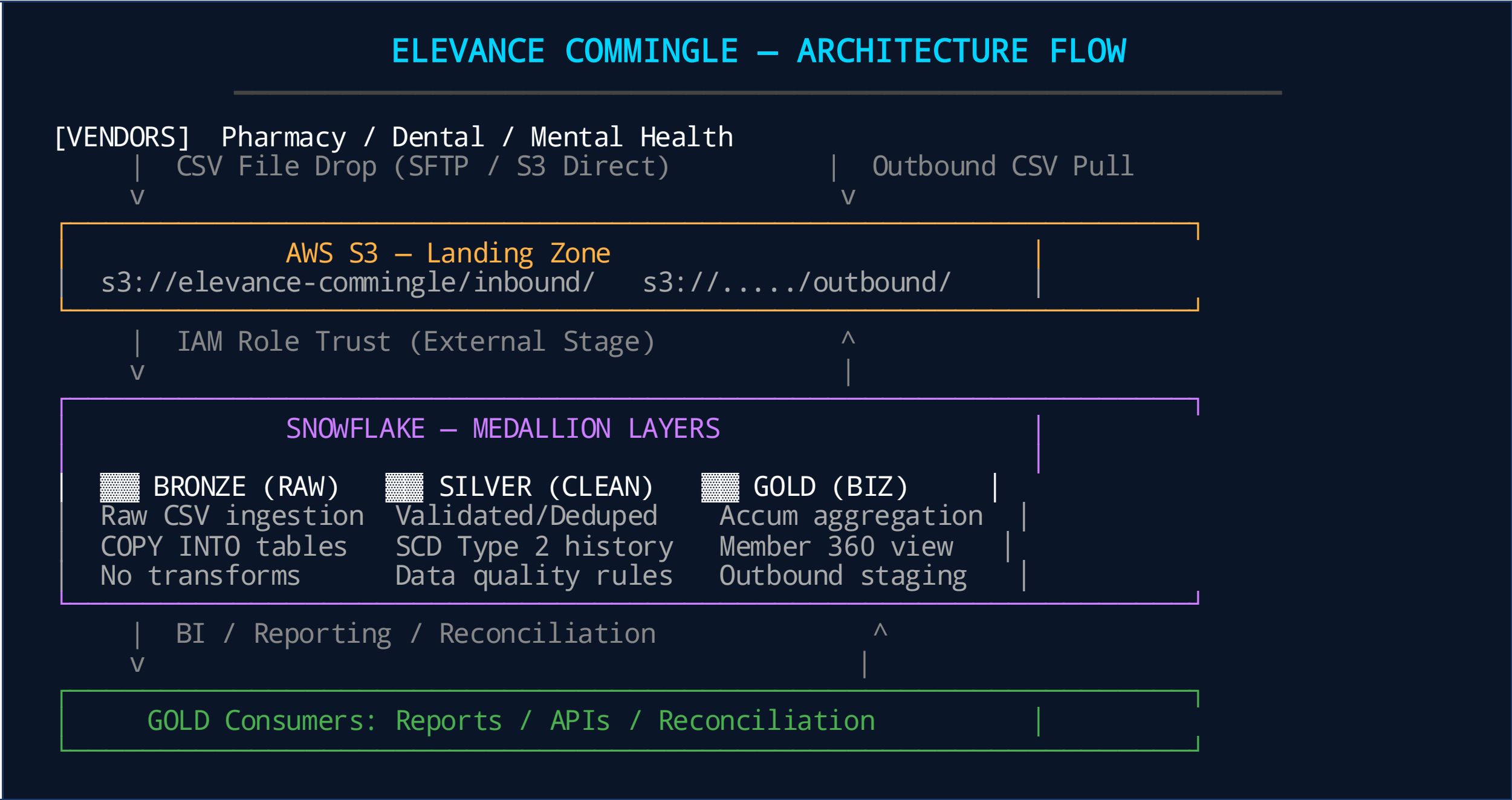
Service Type	Provider	Accum Type	Direction
Pharmacy	External Vendor (e.g., CVS Caremark)	Drug Deductible + OOP	Inbound + Outbound

Service Type	Provider	Accum Type	Direction
Dental	External Vendor (e.g., Delta Dental)	Dental Deductible + OOP	Inbound + Outbound
Mental Health	External Vendor (e.g., Optum Behavioral)	MH Deductible + OOP	Inbound + Outbound
Medical	Elevance Internal	Medical Deductible + OOP	Outbound only

2. Architecture Overview — Medallion on AWS + Snowflake

The platform adopts the Medallion Architecture pattern (Bronze / Silver / Gold) implemented in Snowflake, with AWS S3 serving as the central file landing zone. Data flows through three progressive layers, each adding quality, conformance, and business logic.

2.1 High-Level Architecture Diagram



2.2 Layer Definitions

Layer	Snowflake DB	Purpose	Transform Level	Retention
Bronze	ELEVANCE_BRONZE	Raw ingestion from S3 — no transformation, exact copy of CSV	None	7 years
Silver	ELEVANCE_SILVER	Cleaned, validated, deduplicated, standardized, SCD Type 2	Medium	5 years
Gold	ELEVANCE_GOLD	Business-ready accumulators, member 360, outbound staging	High	3 years

3. AWS Infrastructure Setup

3.1 S3 Bucket Structure

A single S3 bucket with a well-defined prefix structure handles all file movement. Versioning is enabled and lifecycle policies archive files to Glacier after 90 days.

```
s3://elevance-commingle-prod/
├── inbound/                                # Vendor → Elevance (CSV drops)
│   ├── pharmacy/
│   │   ├── raw/YYYY/MM/DD/                # New file landing area
│   │   ├── processed/YYYY/MM/DD/          # After successful COPY INTO
│   │   └── error/YYYY/MM/DD/              # Files that failed validation
│   ├── dental/
│   │   ├── raw/ | processed/ | error/
│   │   └── mental_health/
│   │       ├── raw/ | processed/ | error/
│   └── outbound/                          # Elevance → Vendor (Snowflake export)
│       ├── pharmacy/staging/YYYY/MM/DD/
│       ├── dental/staging/YYYY/MM/DD/
│       └── mental_health/staging/YYYY/MM/DD/
└── audit/                                # File manifests, checksums, SLA logs
    ├── inbound_manifest/
    └── outbound_manifest/
```

3.2 IAM Role Configuration

Snowflake accesses S3 through an IAM Role trust relationship — no access keys or secrets are stored. This is the AWS-recommended approach for Snowflake External Stages.

Step 1 — Create the IAM Policy

```
// IAM Policy: elevance-snowflake-s3-policy
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SnowflakeS3ReadWrite",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:GetObjectVersion",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": [
```

```

        "arn:aws:s3:::elevance-commingle-prod",
        "arn:aws:s3:::elevance-commingle-prod/*"
    ]
}
]
}

```

Step 2 — Create the IAM Role

```

// IAM Role: elevance-snowflake-role
// Trust Policy – initially set principal to your own AWS account,
// then update after running DESC INTEGRATION in Snowflake
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::<SNOWFLAKE_AWS_ACCOUNT_ID>:root"
    },
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": {
        "sts:ExternalId": "<SNOWFLAKE_EXTERNAL_ID>"
      }
    }
  }]
}

```

Step 3 — Create Snowflake Storage Integration

```

-- Run in Snowflake (ACCOUNTADMIN role)
CREATE OR REPLACE STORAGE INTEGRATION elevance_s3_integration
  TYPE = EXTERNAL_STAGE
  STORAGE_PROVIDER = 'S3'
  ENABLED = TRUE
  STORAGE_AWS_ROLE_ARN = 'arn:aws:iam::<AWS_ACCOUNT_ID>:role/elevance-snowflake-role'
  STORAGE_ALLOWED_LOCATIONS = ('s3://elevance-commingle-prod/');

-- Retrieve values to update IAM Trust Policy
DESC INTEGRATION elevance_s3_integration;
-- Copy: STORAGE_AWS_IAM_USER_ARN → update Principal in IAM Trust Policy
-- Copy: STORAGE_AWS_EXTERNAL_ID → update Condition ExternalId in IAM Trust Policy

```

3.3 S3 Event Notifications

S3 Event Notifications trigger an AWS Lambda function whenever a new CSV file lands in any inbound/*/raw/ prefix. The Lambda records file metadata into a Snowflake audit table and optionally triggers a Snowflake Task via the Snowflake REST API.

```

// Lambda function: elevance-inbound-trigger (Python 3.12)

```

```
import boto3, json, snowflake.connector, os

def handler(event, context):
    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key    = record['s3']['object']['key']
        size   = record['s3']['object']['size']
        etag   = record['s3']['object']['eTag']

        # Parse vendor from prefix: inbound/{vendor}/raw/...
        vendor = key.split('/')[1]

        conn = snowflake.connector.connect(
            account = os.environ['SF_ACCOUNT'],
            user    = os.environ['SF_USER'],
            password = os.environ['SF_PASSWORD'],
            warehouse = 'ELEVANCE_ETL_WH',
            database = 'ELEVANCE_BRONZE',
            schema   = 'AUDIT'
        )
        conn.cursor().execute(f'''
            INSERT INTO FILE_LANDING_LOG
            (BUCKET, S3_KEY, VENDOR, FILE_SIZE_BYTES, ETAG, LANDED_AT)
            VALUES ('{bucket}', '{key}', '{vendor}', {size}, '{etag}', CURRENT_TIMESTAMP)
        ''')
        conn.close()
```

4. Snowflake Setup — Databases, Schemas & Objects

4.1 Warehouse & Role Hierarchy

```
-- Warehouses
CREATE WAREHOUSE ELEVANCE_ETL_WH  WAREHOUSE_SIZE='MEDIUM' AUTO_SUSPEND=60 AUTO_RESUME=TRUE;
CREATE WAREHOUSE ELEVANCE_RPTG_WH  WAREHOUSE_SIZE='SMALL'  AUTO_SUSPEND=60 AUTO_RESUME=TRUE;

-- Roles
CREATE ROLE ELEVANCE_ETL_ROLE;      -- Used by pipelines and tasks
CREATE ROLE ELEVANCE_ANALYST_ROLE;  -- Read-only on Gold
CREATE ROLE ELEVANCE_ADMIN_ROLE;    -- Full access

-- Databases (one per medallion layer)
CREATE DATABASE ELEVANCE_BRONZE;
CREATE DATABASE ELEVANCE_SILVER;
CREATE DATABASE ELEVANCE_GOLD;
CREATE DATABASE ELEVANCE_AUDIT;     -- Cross-cutting audit trails
```

4.2 External Stage Definition

```
-- Create external stage pointing to S3 via integration
USE DATABASE ELEVANCE_BRONZE;
USE SCHEMA PUBLIC;

CREATE OR REPLACE STAGE elevance_inbound_stage
  STORAGE_INTEGRATION = elevance_s3_integration
  URL = 's3://elevance-commingle-prod/inbound/'
  FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY = ''
    SKIP_HEADER = 1 NULL_IF = ('NULL','null','')
    EMPTY_FIELD_AS_NULL = TRUE DATE_FORMAT = 'YYYY-MM-DD'
    TIMESTAMP_FORMAT = 'YYYY-MM-DD HH24:MI:SS');

CREATE OR REPLACE STAGE elevance_outbound_stage
  STORAGE_INTEGRATION = elevance_s3_integration
  URL = 's3://elevance-commingle-prod/outbound/';

-- Verify stage can list S3 files
LIST @elevance_inbound_stage;
```

4.3 File Format Objects

```
CREATE OR REPLACE FILE FORMAT csv_inbound_fmt
  TYPE = CSV
  FIELD_DELIMITER = ','
  RECORD_DELIMITER = '\n'
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = ''
```

```
NULL_IF = ( 'NULL','null','','N/A')
EMPTY_FIELD_AS_NULL = TRUE
DATE_FORMAT = 'YYYY-MM-DD'
TIMESTAMP_FORMAT = 'YYYY-MM-DD HH24:MI:SS'
TRIM_SPACE = TRUE
ERROR_ON_COLUMN_COUNT_MISMATCH = FALSE;  -- Log extra columns, don't fail
```

5. Bronze Layer — Raw Ingestion

The Bronze layer stores an exact copy of every CSV file received, with zero business transformation. All columns are VARCHAR to prevent type-mismatch failures. Metadata columns track provenance.

5.1 CSV File Specification — Inbound (Vendor → Elevance)

Each vendor sends a standardized CSV daily by 06:00 ET. The file contains accumulator transactions that occurred the prior business day.

Inbound CSV Filename Convention				
Pattern: {VENDOR_CODE}_{SERVICE_TYPE}_ACCUM_{YYYYMMDD}_{SEQUENCE}.csv				
Example: PHM_PHARMACY_ACCUM_20240315_001.csv				
DNT_DENTAL_ACCUM_20240315_001.csv				
MH_MENTALHEALTH_ACCUM_20240315_001.csv				

#	Column Name	Data Type	Required	Description
1	RECORD_ID	VARCHAR(50)	Y	Vendor-generated unique record identifier
2	MEMBER_ID	VARCHAR(20)	Y	Elevance Member ID (cross-reference key)
3	SUBSCRIBER_ID	VARCHAR(20)	Y	Subscriber/policy holder ID
4	GROUP_NUMBER	VARCHAR(20)	Y	Employer group number
5	PLAN_ID	VARCHAR(20)	Y	Health plan identifier
6	SERVICE_DATE	DATE	Y	Date service was rendered (YYYY-MM-DD)
7	SERVICE_TYPE	VARCHAR(30)	Y	PHARMACY DENTAL MENTAL_HEALTH
8	CLAIM_ID	VARCHAR(50)	Y	Vendor claim/transaction ID
9	CLAIM_STATUS	VARCHAR(20)	Y	PAID DENIED ADJUSTED REVERSED
10	PROVIDER_NPI	VARCHAR(15)	N	National Provider Identifier
11	PROCEDURE_CODE	VARCHAR(20)	N	CPT / NDC / ADA code
12	BILLED_AMOUNT	DECIMAL(12,2)	Y	Total billed by provider
13	ALLOWED_AMOUNT	DECIMAL(12,2)	Y	Contracted allowed amount
14	DEDUCTIBLE_APPLIED	DECIMAL(12,2)	Y	Amount applied toward deductible
15	OOP_APPLIED	DECIMAL(12,2)	Y	Amount applied toward OOP maximum
16	COPAY_AMOUNT	DECIMAL(12,2)	N	Member copay amount
17	COINSURANCE_AMOUNT	DECIMAL(12,2)	N	Member coinsurance amount
18	BENEFIT_PERIOD	VARCHAR(10)	Y	Benefit year: YYYY or YYYY-YYYY
19	ACCUMULATION_TYPE	VARCHAR(20)	Y	INDIVIDUAL FAMILY

#	Column Name	Data Type	Required	Description
20	VENDOR_CODE	VARCHAR(10)	Y	Vendor identifier code
21	VENDOR_FILE_ID	VARCHAR(50)	Y	Source file identifier from vendor
22	CREATED_TIMESTAMP	TIMESTAMP	Y	Record creation timestamp at vendor

5.2 Bronze Table DDL

```
USE DATABASE ELEVANCE_BRONZE;
CREATE SCHEMA IF NOT EXISTS INBOUND;

CREATE OR REPLACE TABLE INBOUND.VENDOR_ACCUM_RAW (
  -- All source columns stored as VARCHAR – no type casting in Bronze
  RECORD_ID          VARCHAR(50),
  MEMBER_ID          VARCHAR(20),
  SUBSCRIBER_ID      VARCHAR(20),
  GROUP_NUMBER       VARCHAR(20),
  PLAN_ID            VARCHAR(20),
  SERVICE_DATE       VARCHAR(20),
  SERVICE_TYPE       VARCHAR(30),
  CLAIM_ID           VARCHAR(50),
  CLAIM_STATUS       VARCHAR(20),
  PROVIDER_NPI        VARCHAR(15),
  PROCEDURE_CODE     VARCHAR(20),
  BILLED_AMOUNT      VARCHAR(20),
  ALLOWED_AMOUNT     VARCHAR(20),
  DEDUCTIBLE_APPLIED VARCHAR(20),
  OOP_APPLIED        VARCHAR(20),
  COPAY_AMOUNT       VARCHAR(20),
  COINSURANCE_AMOUNT VARCHAR(20),
  BENEFIT_PERIOD     VARCHAR(10),
  ACCUMULATION_TYPE  VARCHAR(20),
  VENDOR_CODE        VARCHAR(10),
  VENDOR_FILE_ID     VARCHAR(50),
  CREATED_TIMESTAMP  VARCHAR(30),

  -- Snowflake metadata columns (pipeline-populated)
  _SF_FILE_NAME      VARCHAR(500),
  _SF_FILE_ROW_NUMBER NUMBER,
  _SF_LOAD_TIMESTAMP TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  _SF_LOAD_BATCH_ID  VARCHAR(50),
  _SF_VENDOR_CODE    VARCHAR(10),
  _SF_RECORD_STATUS  VARCHAR(20) DEFAULT 'LOADED', -- LOADED | ERROR
  _SF_ERROR_MESSAGE   VARCHAR(2000)
)
CLUSTER BY (_SF_VENDOR_CODE, SERVICE_DATE);
```

5.3 COPY INTO Command — Inbound Ingestion

```
-- Scheduled via Snowflake Task, runs every 15 minutes
-- Pattern-matched to pick up all pending CSVs
```

```

COPY INTO ELEVANCE_BRONZE.INBOUND.VENDOR_ACCUM_RAW (
  RECORD_ID, MEMBER_ID, SUBSCRIBER_ID, GROUP_NUMBER, PLAN_ID,
  SERVICE_DATE, SERVICE_TYPE, CLAIM_ID, CLAIM_STATUS, PROVIDER_NPI,
  PROCEDURE_CODE, BILLED_AMOUNT, ALLOWED_AMOUNT, DEDUCTIBLE_APPLIED,
  OOP_APPLIED, COPAY_AMOUNT, COINSURANCE_AMOUNT, BENEFIT_PERIOD,
  ACCUMULATION_TYPE, VENDOR_CODE, VENDOR_FILE_ID, CREATED_TIMESTAMP,
  _SF_FILE_NAME, _SF_FILE_ROW_NUMBER, _SF_LOAD_TIMESTAMP, _SF_LOAD_BATCH_ID
)
FROM (
  SELECT
    $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,
    $15,$16,$17,$18,$19,$20,$21,$22,
    METADATA$FILENAME,
    METADATA$FILE_ROW_NUMBER,
    CURRENT_TIMESTAMP(),
    :batch_id      -- bound variable from Task context
  FROM @ELEVANCE_BRONZE.PUBLIC.elevance_inbound_stage
  (FILE_FORMAT => csv_inbound_fmt,
   PATTERN => '.*raw/*.csv')
)
ON_ERROR = CONTINUE      -- Log errors, don't halt batch
PURGE = FALSE;           -- Keep files in S3 (move via Lambda post-load)

```

5.4 Bronze Audit Table

```

USE DATABASE ELEVANCE_AUDIT;
CREATE SCHEMA IF NOT EXISTS BRONZE;

CREATE OR REPLACE TABLE BRONZE.COPY_HISTORY (
  BATCH_ID          VARCHAR(50) PRIMARY KEY,
  VENDOR_CODE       VARCHAR(10),
  S3_FILE_PATH      VARCHAR(1000),
  FILE_NAME         VARCHAR(500),
  FILE_SIZE_BYTES   NUMBER,
  ROWS_SCANNED      NUMBER,
  ROWS_LOADED       NUMBER,
  ROWS_ERRORED      NUMBER,
  LOAD_STATUS       VARCHAR(20), -- SUCCESS | PARTIAL | FAILED
  LOAD_START_TS     TIMESTAMP_NTZ,
  LOAD_END_TS       TIMESTAMP_NTZ,
  DURATION_SECONDS  NUMBER,
  ERROR_DETAILS     VARIANT,      -- JSON array of per-row errors
  CREATED_BY        VARCHAR(100) DEFAULT CURRENT_USER()
);

```

6. Silver Layer — Cleansing, Validation & Conformance

The Silver layer transforms raw Bronze data into a validated, typed, deduplicated, and standardized dataset. It implements SCD Type 2 for full history tracking and applies comprehensive data quality rules.

6.1 Silver Table DDL

```
USE DATABASE ELEVANCE_SILVER;
CREATE SCHEMA IF NOT EXISTS ACCUMS;

CREATE OR REPLACE TABLE ACCUMS.VENDOR_ACCUM_CLEAN (
  -- Surrogate key
  ACCUM_SK          NUMBER AUTOINCREMENT PRIMARY KEY,

  -- Business keys (typed)
  RECORD_ID         VARCHAR(50)    NOT NULL,
  MEMBER_ID         VARCHAR(20)    NOT NULL,
  SUBSCRIBER_ID     VARCHAR(20)    NOT NULL,
  GROUP_NUMBER      VARCHAR(20)    NOT NULL,
  PLAN_ID           VARCHAR(20)    NOT NULL,

  -- Typed & standardized columns
  SERVICE_DATE      DATE            NOT NULL,
  SERVICE_TYPE      VARCHAR(30)    NOT NULL,
  CLAIM_ID          VARCHAR(50)    NOT NULL,
  CLAIM_STATUS      VARCHAR(20)    NOT NULL,
  PROVIDER_NPI      VARCHAR(15),
  PROCEDURE_CODE    VARCHAR(20),
  BILLED_AMOUNT     NUMBER(12,2)   NOT NULL,
  ALLOWED_AMOUNT     NUMBER(12,2)   NOT NULL,
  DEDUCTIBLE_APPLIED NUMBER(12,2)   NOT NULL DEFAULT 0,
  OOP_APPLIED       NUMBER(12,2)   NOT NULL DEFAULT 0,
  COPAY_AMOUNT      NUMBER(12,2)   DEFAULT 0,
  COINSURANCE_AMOUNT NUMBER(12,2)   DEFAULT 0,
  BENEFIT_YEAR      NUMBER(4)      NOT NULL,
  ACCUMULATION_TYPE VARCHAR(20)    NOT NULL,
  VENDOR_CODE       VARCHAR(10)    NOT NULL,

  -- Data quality flags
  DQ_PASSED         BOOLEAN        DEFAULT TRUE,
  DQ_FAILURE_REASONS ARRAY,        -- List of failed rules
  IS_DUPLICATE      BOOLEAN        DEFAULT FALSE,

  -- SCD Type 2 columns
  EFFECTIVE_START_DT DATE          NOT NULL,
  EFFECTIVE_END_DT  DATE,
  IS_CURRENT        BOOLEAN        DEFAULT TRUE,

  -- Lineage
  SOURCE_BATCH_ID   VARCHAR(50),
  SILVER_LOAD_TS    TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
)
CLUSTER BY (VENDOR_CODE, MEMBER_ID, BENEFIT_YEAR);
```

6.2 Data Quality Rules

Rule ID	Column	Check	Action on Fail
DQ-001	MEMBER_ID	Not null, matches pattern ^[A-Z0-9]{8,12}\$	Flag DQ_PASSED=FALSE, log reason
DQ-002	SERVICE_DATE	Valid date, not future, within benefit year	Flag + quarantine
DQ-003	DEDUCTIBLE_APPLIED	Non-negative, ALLOWED_AMOUNT <=	Flag DQ_PASSED=FALSE
DQ-004	OOP_APPLIED	Non-negative, DEDUCTIBLE_APPLIED >=	Flag DQ_PASSED=FALSE
DQ-005	CLAIM_STATUS	One of: PAID, DENIED, ADJUSTED, REVERSED	Standardize or flag
DQ-006	ACCUMULATION_TYPE	INDIVIDUAL or FAMILY	Standardize to uppercase
DQ-007	RECORD_ID	Unique within vendor + benefit_year	Mark IS_DUPLICATE=TRUE
DQ-008	BILLED_AMOUNT	BILLED >= ALLOWED >= 0	Flag + alert vendor
DQ-009	VENDOR_CODE	Matches registered vendor list	Reject — unknown vendor
DQ-010	SERVICE_TYPE	Matches vendor's registered service types	Flag for review

6.3 Silver Transformation SQL

```
-- Bronze → Silver transformation (runs as Snowflake Task)
MERGE INTO ELEVANCE_SILVER.ACCUMS.VENDOR_ACCUM_CLEAN tgt
USING (
  SELECT
    r.RECORD_ID,
    r.MEMBER_ID,
    r.SUBSCRIBER_ID,
    r.GROUP_NUMBER,
    r.PLAN_ID,
    TRY_TO_DATE(r.SERVICE_DATE, 'YYYY-MM-DD')      AS SERVICE_DATE,
    UPPER(TRIM(r.SERVICE_TYPE))                     AS SERVICE_TYPE,
    r.CLAIM_ID,
    UPPER(TRIM(r.CLAIM_STATUS))                     AS CLAIM_STATUS,
    r.PROVIDER_NPI,
    r.PROCEDURE_CODE,
    TRY_TO_NUMBER(r.BILLED_AMOUNT, 12, 2)           AS BILLED_AMOUNT,
    TRY_TO_NUMBER(r.ALLOWED_AMOUNT, 12, 2)           AS ALLOWED_AMOUNT,
    NVL(TRY_TO_NUMBER(r.DEDUCTIBLE_APPLIED, 12, 2), 0) AS DEDUCTIBLE_APPLIED,
    NVL(TRY_TO_NUMBER(r.OOP_APPLIED, 12, 2), 0)      AS OOP_APPLIED,
    NVL(TRY_TO_NUMBER(r.COPAY_AMOUNT, 12, 2), 0)     AS COPAY_AMOUNT,
    NVL(TRY_TO_NUMBER(r.COINSURANCE_AMOUNT, 12, 2), 0) AS COINSURANCE_AMOUNT,
    LEFT(r.BENEFIT_PERIOD, 4)::NUMBER               AS BENEFIT_YEAR,
    UPPER(TRIM(r.ACCUMULATION_TYPE))                AS ACCUMULATION_TYPE,
    r.VENDOR_CODE,
    r._SF_LOAD_BATCH_ID                             AS SOURCE_BATCH_ID,
    -- Data quality flags computed inline
```

```
CASE WHEN r.MEMBER_ID IS NULL THEN FALSE
      WHEN TRY_TO_DATE(r.SERVICE_DATE,'YYYY-MM-DD') IS NULL THEN FALSE
      WHEN TRY_TO_NUMBER(r.BILLED_AMOUNT,12,2) < 0 THEN FALSE
      ELSE TRUE END
      AS DQ_PASSED,
CURRENT_DATE()
      AS EFFECTIVE_START_DT
FROM ELEVANCE_BRONZE.INBOUND.VENDOR_ACCUM_RAW r
WHERE r._SF_RECORD_STATUS = 'LOADED'
      AND r._SF_LOAD_BATCH_ID = :batch_id
) src
ON tgt.RECORD_ID = src.RECORD_ID
AND tgt.VENDOR_CODE = src.VENDOR_CODE
AND tgt.IS_CURRENT = TRUE

-- SCD Type 2: close existing record if changed
WHEN MATCHED AND (
  tgt.CLAIM_STATUS <> src.CLAIM_STATUS OR
  tgt.DEDUCTIBLE_APPLIED <> src.DEDUCTIBLE_APPLIED OR
  tgt.OOP_APPLIED <> src.OOP_APPLIED
) THEN UPDATE SET
  tgt.EFFECTIVE_END_DT = CURRENT_DATE() - 1,
  tgt.IS_CURRENT = FALSE

-- New records and new SCD2 versions
WHEN NOT MATCHED THEN INSERT (
  RECORD_ID, MEMBER_ID, SUBSCRIBER_ID, GROUP_NUMBER, PLAN_ID,
  SERVICE_DATE, SERVICE_TYPE, CLAIM_ID, CLAIM_STATUS, PROVIDER_NPI,
  PROCEDURE_CODE, BILLED_AMOUNT, ALLOWED_AMOUNT, DEDUCTIBLE_APPLIED,
  OOP_APPLIED, COPAY_AMOUNT, COINSURANCE_AMOUNT, BENEFIT_YEAR,
  ACCUMULATION_TYPE, VENDOR_CODE, DQ_PASSED, EFFECTIVE_START_DT,
  IS_CURRENT, SOURCE_BATCH_ID)
VALUES (
  src.RECORD_ID, src.MEMBER_ID, src.SUBSCRIBER_ID, src.GROUP_NUMBER,
  src.PLAN_ID, src.SERVICE_DATE, src.SERVICE_TYPE, src.CLAIM_ID,
  src.CLAIM_STATUS, src.PROVIDER_NPI, src.PROCEDURE_CODE,
  src.BILLED_AMOUNT, src.ALLOWED_AMOUNT, src.DEDUCTIBLE_APPLIED,
  src.OOP_APPLIED, src.COPAY_AMOUNT, src.COINSURANCE_AMOUNT,
  src.BENEFIT_YEAR, src.ACCUMULATION_TYPE, src.VENDOR_CODE,
  src.DQ_PASSED, src.EFFECTIVE_START_DT, TRUE, src.SOURCE_BATCH_ID);
```

7. Gold Layer — Business-Ready Accumulators

The Gold layer aggregates cleansed Silver data into actionable member accumulator views. It serves as the single source of truth for member Deductible and OOP balances and provides outbound staging tables for vendor data feeds.

7.1 Member Accumulator Summary Table

```
USE DATABASE ELEVANCE_GOLD;
CREATE SCHEMA IF NOT EXISTS ACCUMS;

CREATE OR REPLACE TABLE ACCUMS.MEMBER_ACCUM_SUMMARY (
  MEMBER_ACCUM_SK          NUMBER AUTOINCREMENT PRIMARY KEY,
  MEMBER_ID                VARCHAR(20) NOT NULL,
  SUBSCRIBER_ID            VARCHAR(20) NOT NULL,
  GROUP_NUMBER             VARCHAR(20) NOT NULL,
  PLAN_ID                  VARCHAR(20) NOT NULL,
  BENEFIT_YEAR             NUMBER(4) NOT NULL,
  ACCUMULATION_TYPE        VARCHAR(20) NOT NULL, -- INDIVIDUAL | FAMILY

  -- Deductible tracking by service type
  MEDICAL_DEDUCTIBLE_ACCUM NUMBER(12,2) DEFAULT 0,
  PHARMACY_DEDUCTIBLE_ACCUM NUMBER(12,2) DEFAULT 0,
  DENTAL_DEDUCTIBLE_ACCUM  NUMBER(12,2) DEFAULT 0,
  MH_DEDUCTIBLE_ACCUM      NUMBER(12,2) DEFAULT 0,
  TOTAL_DEDUCTIBLE_ACCUM   NUMBER(12,2) DEFAULT 0,
  DEDUCTIBLE_LIMIT         NUMBER(12,2), -- From plan config
  DEDUCTIBLE_MET           BOOLEAN DEFAULT FALSE,

  -- OOP tracking by service type
  MEDICAL_OOP_ACCUM        NUMBER(12,2) DEFAULT 0,
  PHARMACY_OOP_ACCUM       NUMBER(12,2) DEFAULT 0,
  DENTAL_OOP_ACCUM         NUMBER(12,2) DEFAULT 0,
  MH_OOP_ACCUM             NUMBER(12,2) DEFAULT 0,
  TOTAL_OOP_ACCUM          NUMBER(12,2) DEFAULT 0,
  OOP_LIMIT                NUMBER(12,2), -- From plan config
  OOP_MET                  BOOLEAN DEFAULT FALSE,

  -- Remaining balances
  DEDUCTIBLE_REMAINING     NUMBER(12,2),
  OOP_REMAINING            NUMBER(12,2),

  -- Metadata
  LAST_UPDATED_TS          TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
  LAST_SERVICE_DATE        DATE,
  RECORD_COUNT             NUMBER DEFAULT 0
)
CLUSTER BY (MEMBER_ID, BENEFIT_YEAR);
```

7.2 Gold Aggregation SQL

```
-- Refresh Gold accumulator summary from Silver (full recalc per member)
```

```

MERGE INTO ELEVANCE_GOLD.ACCUMS.MEMBER_ACCUM_SUMMARY tgt
USING (
  SELECT
    MEMBER_ID, SUBSCRIBER_ID, GROUP_NUMBER, PLAN_ID,
    BENEFIT_YEAR, ACCUMULATION_TYPE,
    SUM(CASE WHEN SERVICE_TYPE='MEDICAL' THEN DEDUCTIBLE_APPLIED ELSE 0 END) AS
MEDICAL_DED,
    SUM(CASE WHEN SERVICE_TYPE='PHARMACY' THEN DEDUCTIBLE_APPLIED ELSE 0 END) AS
PHARM_DED,
    SUM(CASE WHEN SERVICE_TYPE='DENTAL' THEN DEDUCTIBLE_APPLIED ELSE 0 END) AS
DENTAL_DED,
    SUM(CASE WHEN SERVICE_TYPE='MENTAL_HEALTH' THEN DEDUCTIBLE_APPLIED ELSE 0 END) AS MH_DED,
    SUM(DEDUCTIBLE_APPLIED) AS TOTAL_DED,
    SUM(CASE WHEN SERVICE_TYPE='MEDICAL' THEN OOP_APPLIED ELSE 0 END) AS MEDICAL_OOP,
    SUM(CASE WHEN SERVICE_TYPE='PHARMACY' THEN OOP_APPLIED ELSE 0 END) AS PHARM_OOP,
    SUM(CASE WHEN SERVICE_TYPE='DENTAL' THEN OOP_APPLIED ELSE 0 END) AS DENTAL_OOP,
    SUM(CASE WHEN SERVICE_TYPE='MENTAL_HEALTH' THEN OOP_APPLIED ELSE 0 END) AS MH_OOP,
    SUM(OOP_APPLIED) AS TOTAL_OOP,
    MAX(SERVICE_DATE) AS LAST_SVC_DATE,
    COUNT(1) AS REC_COUNT
  FROM ELEVANCE_SILVER.ACCUMS.VENDOR_ACCUM_CLEAN
  WHERE IS_CURRENT = TRUE AND DQ_PASSED = TRUE
    AND CLAIM_STATUS IN ('PAID', 'ADJUSTED')
  GROUP BY 1,2,3,4,5,6
) src
ON tgt.MEMBER_ID = src.MEMBER_ID
AND tgt.BENEFIT_YEAR = src.BENEFIT_YEAR
AND tgt.ACCUMULATION_TYPE = src.ACCUMULATION_TYPE
WHEN MATCHED THEN UPDATE SET
  tgt.MEDICAL_DEDUCTIBLE_ACCUM = src.MEDICAL_DED,
  tgt.PHARMACY_DEDUCTIBLE_ACCUM = src.PHARM_DED,
  tgt.DENTAL_DEDUCTIBLE_ACCUM = src.DENTAL_DED,
  tgt.MH_DEDUCTIBLE_ACCUM = src.MH_DED,
  tgt.TOTAL_DEDUCTIBLE_ACCUM = src.TOTAL_DED,
  tgt.MEDICAL_OOP_ACCUM = src.MEDICAL_OOP,
  tgt.PHARMACY_OOP_ACCUM = src.PHARM_OOP,
  tgt.DENTAL_OOP_ACCUM = src.DENTAL_OOP,
  tgt.MH_OOP_ACCUM = src.MH_OOP,
  tgt.TOTAL_OOP_ACCUM = src.TOTAL_OOP,
  tgt.DEDUCTIBLE_REMAINING = GREATEST(tgt.DEDUCTIBLE_LIMIT - src.TOTAL_DED, 0),
  tgt.OOP_REMAINING = GREATEST(tgt.OOP_LIMIT - src.TOTAL_OOP, 0),
  tgt.DEDUCTIBLE_MET = (src.TOTAL_DED >= tgt.DEDUCTIBLE_LIMIT),
  tgt.OOP_MET = (src.TOTAL_OOP >= tgt.OOP_LIMIT),
  tgt.LAST_UPDATED_TS = CURRENT_TIMESTAMP(),
  tgt.LAST_SERVICE_DATE = src.LAST_SVC_DATE,
  tgt.RECORD_COUNT = src.REC_COUNT
WHEN NOT MATCHED THEN INSERT (...) VALUES (...); -- same columns

```

8. Outbound Process — Elevance → Vendors

The Outbound process extracts medical accumulator data from Elevance's Gold layer and generates vendor-specific CSV files, which are deposited to S3 and delivered to vendors via SFTP or direct S3 access. This keeps vendors informed of any medical claims adjudicated by Elevance that affect a member's shared deductible or OOP.

8.1 Outbound CSV File Specification

Outbound CSV Filename Convention	
Pattern: ELEVANCE_{VENDOR_CODE}_ACCUM_{YYYYMMDD}_{SEQUENCE}.csv	
Example: ELEVANCE_PHM_ACCUM_20240315_001.csv	
ELEVANCE_DNT_ACCUM_20240315_001.csv	

#	Column Name	Data Type	Description
1	RECORD_ID	VARCHAR(50)	Elevance-generated unique record ID (UUID)
2	MEMBER_ID	VARCHAR(20)	Elevance Member ID
3	SUBSCRIBER_ID	VARCHAR(20)	Subscriber/policy holder ID
4	GROUP_NUMBER	VARCHAR(20)	Employer group number
5	PLAN_ID	VARCHAR(20)	Health plan identifier
6	SERVICE_DATE	DATE	Date medical service was rendered
7	CLAIM_ID	VARCHAR(50)	Elevance internal claim ID
8	CLAIM_STATUS	VARCHAR(20)	PAID ADJUSTED REVERSED
9	PROCEDURE_CODE	VARCHAR(20)	CPT procedure code
10	BILLED_AMOUNT	DECIMAL(12,2)	Total billed amount
11	ALLOWED_AMOUNT	DECIMAL(12,2)	Allowed amount after contracting
12	MEDICAL_DEDUCTIBLE_APPLIED	DECIMAL(12,2)	Medical deductible applied this claim
13	MEDICAL_OOP_APPLIED	DECIMAL(12,2)	Medical OOP applied this claim
14	YTD_MEDICAL_DEDUCTIBLE	DECIMAL(12,2)	Running YTD medical deductible total
15	YTD_MEDICAL_OOP	DECIMAL(12,2)	Running YTD medical OOP total
16	BENEFIT_YEAR	NUMBER(4)	Benefit year (YYYY)
17	ACCUMULATION_TYPE	VARCHAR(20)	INDIVIDUAL FAMILY
18	FILE_GENERATED_TS	TIMESTAMP	When Elevance generated this file

8.2 Outbound Gold Staging Table

```

USE DATABASE ELEVANCE_GOLD;
CREATE SCHEMA IF NOT EXISTS OUTBOUND;

CREATE OR REPLACE TABLE OUTBOUND.VENDOR_OUTBOUND_STAGING (
  RECORD_ID          VARCHAR(50) DEFAULT UUID_STRING(),
  MEMBER_ID          VARCHAR(20) NOT NULL,
  SUBSCRIBER_ID      VARCHAR(20),
  GROUP_NUMBER       VARCHAR(20),
  PLAN_ID            VARCHAR(20),
  SERVICE_DATE       DATE,
  CLAIM_ID           VARCHAR(50),
  CLAIM_STATUS       VARCHAR(20),
  PROCEDURE_CODE     VARCHAR(20),
  BILLED_AMOUNT      NUMBER(12,2),
  ALLOWED_AMOUNT      NUMBER(12,2),
  MEDICAL_DEDUCTIBLE_APPLIED NUMBER(12,2),
  MEDICAL_OOP_APPLIED NUMBER(12,2),
  YTD_MEDICAL_DEDUCTIBLE NUMBER(12,2),
  YTD_MEDICAL_OOP    NUMBER(12,2),
  BENEFIT_YEAR       NUMBER(4),
  ACCUMULATION_TYPE  VARCHAR(20),
  TARGET_VENDOR_CODE VARCHAR(10) NOT NULL,
  FILE_GENERATED_TS   TIMESTAMP_NTZ,
  OUTBOUND_FILE_NAME  VARCHAR(500),
  OUTBOUND_STATUS     VARCHAR(20) DEFAULT 'PENDING', -- PENDING|EXPORTED|DELIVERED
  EXPORT_BATCH_ID     VARCHAR(50)
);

```

8.3 Outbound COPY INTO S3

```

-- Export Gold staging to S3 for each vendor
-- Run per vendor via Task chain

COPY INTO @ELEVANCE_BRONZE.PUBLIC.elevance_outbound_stage/pharmacy/staging/
FROM (
  SELECT
    RECORD_ID, MEMBER_ID, SUBSCRIBER_ID, GROUP_NUMBER, PLAN_ID,
    SERVICE_DATE::VARCHAR, CLAIM_ID, CLAIM_STATUS, PROCEDURE_CODE,
    BILLED_AMOUNT, ALLOWED_AMOUNT, MEDICAL_DEDUCTIBLE_APPLIED,
    MEDICAL_OOP_APPLIED, YTD_MEDICAL_DEDUCTIBLE, YTD_MEDICAL_OOP,
    BENEFIT_YEAR, ACCUMULATION_TYPE, CURRENT_TIMESTAMP()::VARCHAR
  FROM ELEVANCE_GOLD.OUTBOUND.VENDOR_OUTBOUND_STAGING
  WHERE TARGET_VENDOR_CODE = 'PHM'
     AND OUTBOUND_STATUS = 'PENDING'
     AND EXPORT_BATCH_ID = :batch_id
)
FILE_FORMAT = (TYPE = CSV FIELD_OPTIONALLY_ENCLOSED_BY = ''
               NULL_IF = () EMPTY_FIELD_AS_NULL = FALSE)
HEADER = TRUE
SINGLE = FALSE
MAX_FILE_SIZE = 104857600 -- 100MB per file
OVERWRITE = FALSE;

```

9. Snowflake Task Orchestration

Snowflake Tasks form a DAG (Directed Acyclic Graph) orchestrating the full pipeline. The root task polls for new files; child tasks handle Bronze ingestion, Silver transformation, Gold aggregation, and Outbound export in sequence.

9.1 Task DAG Structure

```
TASK DAG: ELEVANCE_COMMINGLE_ROOT
├── T1_ROOT_POLL          (every 15 min) Check for new S3 files
│   ├── T2_BRONZE_LOAD    Load new CSVs → Bronze (COPY INTO)
│   │   ├── T3_BRONZE_AUDIT Record load stats → Audit DB
│   │   │   ├── T4_SILVER_TRANSFORM Bronze → Silver MERGE
│   │   │   │   ├── T5_SILVER_DQ_CHECK Run DQ assertions
│   │   │   │   └── T6_GOLD_REFRESH    Silver → Gold MERGE
│   │   │       ├── T7_OUTBOUND_STAGE Build outbound CSV staging
│   │   │       └── T8_OUTBOUND_EXPORT COPY INTO S3 outbound stage
│   │           └── T9_NOTIFY          SNS alert on completion
└── T10_RECONCILIATION    (daily 08:00 ET) Vendor vs Elevance accum comparison
```

9.2 Task Definitions

```
-- Root poll task
CREATE OR REPLACE TASK T1_ROOT_POLL
  WAREHOUSE = ELEVANCE_ETL_WH
  SCHEDULE = '15 MINUTES'
  COMMENT = 'Root task: check for new inbound files and trigger pipeline'
AS
  CALL ELEVANCE_BRONZE.PUBLIC.SP_CHECK_NEW_FILES();

-- Bronze load task (child of T1)
CREATE OR REPLACE TASK T2_BRONZE_LOAD
  WAREHOUSE = ELEVANCE_ETL_WH
  AFTER T1_ROOT_POLL
  WHEN SYSTEM$STREAM_HAS_DATA('ELEVANCE_BRONZE.INBOUND.NEW_FILES_STREAM')
AS
  CALL ELEVANCE_BRONZE.PUBLIC.SP_LOAD_BRONZE(:batch_id);

-- Silver transform (child of T3)
CREATE OR REPLACE TASK T4_SILVER_TRANSFORM
  WAREHOUSE = ELEVANCE_ETL_WH
  AFTER T3_BRONZE_AUDIT
AS
  CALL ELEVANCE_SILVER.ACCUMS.SP_TRANSFORM_SILVER(:batch_id);

-- Enable all tasks (start from leaves up to root)
ALTER TASK T9_NOTIFY RESUME;
ALTER TASK T8_OUTBOUND_EXPORT RESUME;
-- ... (resume all child tasks first)
ALTER TASK T1_ROOT_POLL RESUME; -- Start root last
```

10. Reconciliation & Exception Handling

10.1 Daily Reconciliation Process

Each morning, a reconciliation process compares Elevance's Gold accumulator totals against vendor-reported totals from the prior day's inbound file. Discrepancies above threshold trigger alerts and exception workflows.

```
-- Daily reconciliation query
CREATE OR REPLACE TABLE ELEVANCE_GOLD.RECON.DAILY_RECONCILIATION AS
SELECT
  g.MEMBER_ID,
  g.BENEFIT_YEAR,
  g.ACCUMULATION_TYPE,
  g.TOTAL_DEDUCTIBLE_ACCUM          AS ELEVANCE_DEDUCTIBLE,
  v.VENDOR_REPORTED_DEDUCTIBLE      AS VENDOR_DEDUCTIBLE,
  ABS(g.TOTAL_DEDUCTIBLE_ACCUM
      - v.VENDOR_REPORTED_DEDUCTIBLE) AS DED_VARIANCE,
  g.TOTAL_OOP_ACCUM                 AS ELEVANCE_OOP,
  v.VENDOR_REPORTED_OOP             AS VENDOR_OOP,
  ABS(g.TOTAL_OOP_ACCUM
      - v.VENDOR_REPORTED_OOP)      AS OOP_VARIANCE,
  CASE
    WHEN ABS(g.TOTAL_DEDUCTIBLE_ACCUM - v.VENDOR_REPORTED_DEDUCTIBLE) > 1.00
      OR ABS(g.TOTAL_OOP_ACCUM - v.VENDOR_REPORTED_OOP) > 1.00
    THEN 'EXCEPTION'
    ELSE 'RECONCILED'
  END AS RECON_STATUS,
  CURRENT_TIMESTAMP() AS RECON_TS
FROM ELEVANCE_GOLD.ACCUMS.MEMBER_ACCUM_SUMMARY g
JOIN ELEVANCE_GOLD.RECON.VENDOR_REPORTED_ACCUMS v
  ON g.MEMBER_ID = v.MEMBER_ID
 AND g.BENEFIT_YEAR = v.BENEFIT_YEAR
 AND g.ACCUMULATION_TYPE = v.ACCUMULATION_TYPE;
```

10.2 Exception Management

Exception Type	Threshold	Auto-Action	Manual Escalation
Deductible Variance	> \$1.00 per member	Flag in recon table, notify vendor via email	Within 24h if unresolved
OOP Variance	> \$1.00 per member	Flag in recon table, notify vendor	Within 24h if unresolved
Missing Inbound File	File not received by 08:00 ET	SNS alert to ops team	Vendor contact within 2h
DQ Failure Rate	> 5% rows failing DQ	Pause pipeline, quarantine batch	Immediate ops escalation
Duplicate Records	Same RECORD_ID from	Mark IS_DUPLICATE=TRUE, exclude from Gold	Log for vendor feedback

Exception Type	Threshold	Auto-Action	Manual Escalation
	same vendor		
Unknown Member	MEMBER_ID not in Elevance master	Quarantine + alert	Manual lookup required

11. Security & Data Governance

11.1 Security Architecture

- IAM Role-based S3 access — no access keys stored anywhere
- Snowflake Network Policy restricts connections to approved IP ranges
- All data encrypted at rest (S3: SSE-S3, Snowflake: AES-256)
- All data encrypted in transit (TLS 1.2+)
- PHI/PII fields masked in Silver/Gold using Snowflake Dynamic Data Masking
- Snowflake column-level security policies applied to MEMBER_ID, SUBSCRIBER_ID
- Audit logging enabled via Snowflake ACCESS_HISTORY and QUERY_HISTORY
- S3 CloudTrail logging for all bucket access events

11.2 Dynamic Data Masking — PII Protection

```
-- Masking policy: non-ETL roles see masked member IDs
CREATE OR REPLACE MASKING POLICY mask_member_id
AS (val VARCHAR) RETURNS VARCHAR ->
CASE
  WHEN CURRENT_ROLE() IN ('ELEVANCE_ETL_ROLE', 'ELEVANCE_ADMIN_ROLE')
  THEN val
  ELSE CONCAT('XXX-', RIGHT(val, 4)) -- Show last 4 only
END;

-- Apply to Silver and Gold tables
ALTER TABLE ELEVANCE_SILVER.ACCUMS.VENDOR_ACCUM_CLEAN
  MODIFY COLUMN MEMBER_ID SET MASKING POLICY mask_member_id;

ALTER TABLE ELEVANCE_GOLD.ACCUMS.MEMBER_ACCUM_SUMMARY
  MODIFY COLUMN MEMBER_ID SET MASKING POLICY mask_member_id;
```

11.3 HIPAA Compliance Controls

HIPAA Safeguard	Control Implemented	Evidence
Access Control	RBAC via Snowflake roles; IAM policies on S3	Role grants documented; reviewed quarterly
Audit Controls	Snowflake ACCESS_HISTORY; S3 CloudTrail	90-day log retention; SIEM integration
Integrity	File checksums (MD5/SHA256) validated on ingest	Checksum mismatch = automatic rejection
Transmission Security	TLS 1.2+ for all data in transit	Certificate monitoring in place
Minimum Necessary	Analysts see masked PII; ETL sees full data	Masking policies applied per role
Data Retention	7-year Bronze retention; defined Silver/Gold	Snowflake lifecycle + S3 Glacier archive

HIPAA Safeguard	Control Implemented	Evidence
	TTL	

12. Monitoring, SLA & Alerting

12.1 SLA Definitions

Process	SLA	Measurement	Alert Threshold
Inbound file receipt	06:00 ET daily	File lands in S3 inbound/raw/	Breach if not received by 07:00 ET
Bronze complete load	Within 30 min of file receipt	COPY INTO completion timestamp	Alert if > 45 minutes
Silver complete transform	Within 60 min of file receipt	Silver MERGE completion timestamp	Alert if > 90 minutes
Gold complete refresh	Within 90 min of file receipt	Gold table LAST_UPDATED_TS	Alert if > 2 hours
Outbound delivery file	By 12:00 ET daily	File available in S3 outbound/	Alert if not delivered by 11:00 ET
Reconciliation complete	By 09:00 ET daily	Recon table populated	Alert if not complete by 09:30 ET

12.2 Key Monitoring Queries

```
-- Pipeline health dashboard query
SELECT
  DATE_TRUNC('DAY', LOAD_START_TS) AS LOAD_DATE,
  VENDOR_CODE,
  COUNT(*) AS TOTAL_FILES,
  SUM(ROWS_LOADED) AS TOTAL_ROWS_LOADED,
  SUM(ROWS_ERRORED) AS TOTAL_ROWS_ERRORED,
  ROUND(SUM(ROWS_ERRORED)*100.0/NULLIF(SUM(ROWS_SCANNED),0),2) AS ERROR_PCT,
  AVG(DURATION_SECONDS) AS AVG_LOAD_DURATION_SEC,
  MAX(DURATION_SECONDS) AS MAX_LOAD_DURATION_SEC,
  COUNT(CASE WHEN LOAD_STATUS='FAILED' THEN 1 END) AS FAILED_FILES
FROM ELEVANCE_AUDIT.BRONZE.COPY_HISTORY
WHERE LOAD_START_TS >= DATEADD('DAY', -7, CURRENT_DATE())
GROUP BY 1, 2
ORDER BY 1 DESC, 2;
```

12.3 Alerting Setup

- AWS SNS topics: elevance-pipeline-alerts, elevance-sla-breach, elevance-dq-alerts
- Subscribers: DataOps team (PagerDuty), Vendor Relations (email), Compliance (email)
- Snowflake ALERT objects trigger on DQ failure rate > 5%
- CloudWatch alarms monitor Lambda execution failures and S3 event throughput
- Daily pipeline summary email generated by Snowflake Task at 09:00 ET

13. End-to-End Flow Summary

13.1 Inbound Flow — Step by Step

Step	Actor	Action	Output
1	External Vendor	Generates daily CSV of member accum transactions	CSV file ready for delivery
2	External Vendor	Drops CSV to S3 s3://...../inbound/{vendor}/raw/YYYY/MM/DD/	File in S3 landing zone
3	S3 Event + Lambda	S3 trigger fires Lambda; Lambda logs to Audit DB	FILE_LANDING_LOG entry created
4	Snowflake Task T1	Root task polls for new files via stream	Pipeline triggered
5	Task T2 (Bronze)	COPY INTO Bronze table from stage; all columns VARCHAR	Bronze.VENDOR_ACCUM_RAW populated
6	Task T3 (Audit)	Record batch stats in AUDIT.BRONZE.COPY_HISTORY	Audit trail created
7	Lambda (post-load)	Move processed CSV from raw/ → processed/ in S3	S3 file archived
8	Task T4 (Silver)	MERGE Bronze → Silver with typing, DQ, SCD2	Silver.VENDOR_ACCUM_CLEAN updated
9	Task T5 (DQ)	Run DQ assertion checks; alert on failures > 5%	DQ report in Audit DB
10	Task T6 (Gold)	Aggregate Silver → Gold member accum summary	Gold.MEMBER_ACCUM_SUMMARY updated
11	Task T9	Send SNS completion notification to ops	Success alert sent

13.2 Outbound Flow — Step by Step

Step	Actor	Action	Output
1	Task T7	Query Gold; build vendor-specific outbound records in staging table	OUTBOUND.VENDOR_OUTBOUND_STAGING populated
2	Task T8	COPY INTO S3 outbound stage, per vendor, with CSV header	CSV in s3://...../outbound/{vendor}/
3	Lambda (trigger)	S3 event fires; Lambda updates staging table OUTBOUND_STATUS=EXPORTED	Status updated in Gold
4	SFTP/S3 Delivery	Vendor pulls file from S3 or Elevance pushes via SFTP	Vendor receives medical accum CSV
5	Vendor ACK	Vendor sends acknowledgement file back to inbound/ack/	Confirmation logged in Audit DB

Step	Actor	Action	Output
6	Reconciliation	Daily recon compares Gold totals vs vendor-reported totals	RECON.DAILY_RECONCILIATION updated

14. Glossary

Term	Definition
Accumulator (Accum)	Running total of amounts applied toward a member's deductible or OOP maximum within a benefit period
Deductible	Amount a member must pay out-of-pocket for covered services before insurance begins to pay
Out-of-Pocket Maximum (OOP)	Maximum total a member pays in a benefit year; after this limit, plan covers 100% of covered services
Commingle	Process of combining/synchronizing medical and non-medical accumulator data across internal and external systems
Bronze Layer	Raw ingestion layer in Medallion Architecture; stores exact CSV data with no transformation
Silver Layer	Cleansed, validated, typed, and deduplicated layer; implements SCD Type 2 for history
Gold Layer	Business-ready layer with aggregated accumulators, member 360 views, and outbound staging
SCD Type 2	Slowly Changing Dimension Type 2 — tracks full history of changes with effective dates
IAM Role	AWS Identity and Access Management Role used to grant Snowflake access to S3 without storing credentials
External Stage	Snowflake object pointing to an external location (S3) from which data can be loaded
COPY INTO	Snowflake command to bulk-load files from a stage into a table
Medallion Architecture	Data lakehouse pattern with Bronze/Silver/Gold layers for progressive data quality
HIPAA	Health Insurance Portability and Accountability Act — US law governing PHI privacy and security
PHI	Protected Health Information — individually identifiable health data protected under HIPAA
NPI	National Provider Identifier — unique 10-digit number for healthcare providers
DQ	Data Quality — rules and checks ensuring data meets defined standards
SLA	Service Level Agreement — committed time targets for pipeline completion
SNS	Amazon Simple Notification Service — AWS messaging service used for pipeline alerts

15. Appendix — Quick Reference Cheat Sheet

Component	Key Detail
-----------	------------

Component	Key Detail
S3 Bucket	s3://elevance-commingle-prod/
IAM Role ARN	arn:aws:iam::<ACCOUNT>:role/elevance-snowflake-role
Snowflake Integration	elevance_s3_integration
Inbound Stage	@ELEVANCE_BRONZE.PUBLIC.elevance_inbound_stage
Outbound Stage	@ELEVANCE_BRONZE.PUBLIC.elevance_outbound_stage
Bronze DB	ELEVANCE_BRONZE.INBOUND.VENDOR_ACCUM_RAW
Silver DB	ELEVANCE_SILVER.ACCUMS.VENDOR_ACCUM_CLEAN
Gold DB	ELEVANCE_GOLD.ACCUMS.MEMBER_ACCUM_SUMMARY
Gold Outbound	ELEVANCE_GOLD.OUTBOUND.VENDOR_OUTBOUND_STAGING
Audit DB	ELEVANCE_AUDIT.BRONZE.COPY_HISTORY
ETL Warehouse	ELEVANCE_ETL_WH (MEDIUM, auto-suspend 60s)
Root Task	T1_ROOT_POLL — every 15 minutes
Vendor Codes	PHM=Pharmacy DNT=Dental MH=Mental Health
Inbound SLA	File received by 06:00 ET, Gold updated by 07:30 ET
Outbound SLA	Delivered to vendors by 12:00 ET daily